极客时间

Sub-Agents 角色分工

给 AI 分工——让每个角色各司其职

第 1 周 · 第 3 节

讲师：黄佳

极客时间黄佳老师出品

极客时间

回顾：SDD 第一层完成

- **AGENTS.md** — SDD 三层规范的第一层：项目规范
- **6 个维度** — 项目概述、技术栈、编码规范、项目结构、工作流程、特殊约束
- **声明式配置** — 描述「要什么」，让 Agent 自己决定怎么做
-
- **今天：SDD 第二层——角色规范**

AGENTS.md 里定义了多个 Agent 角色，可怎么让 AI 真正“变成”这些角色？

——答案就是 Sub-Agent 角色文件。

极客时间黄佳老师出品

极客时间

核心概念

极客时间

Agent = 角色 + 权限 + 任务

极客时间

一个 Sub-Agent 就是一个被限定了角色、权限和任务范围的 AI 实例

极客时间黄佳老师出品

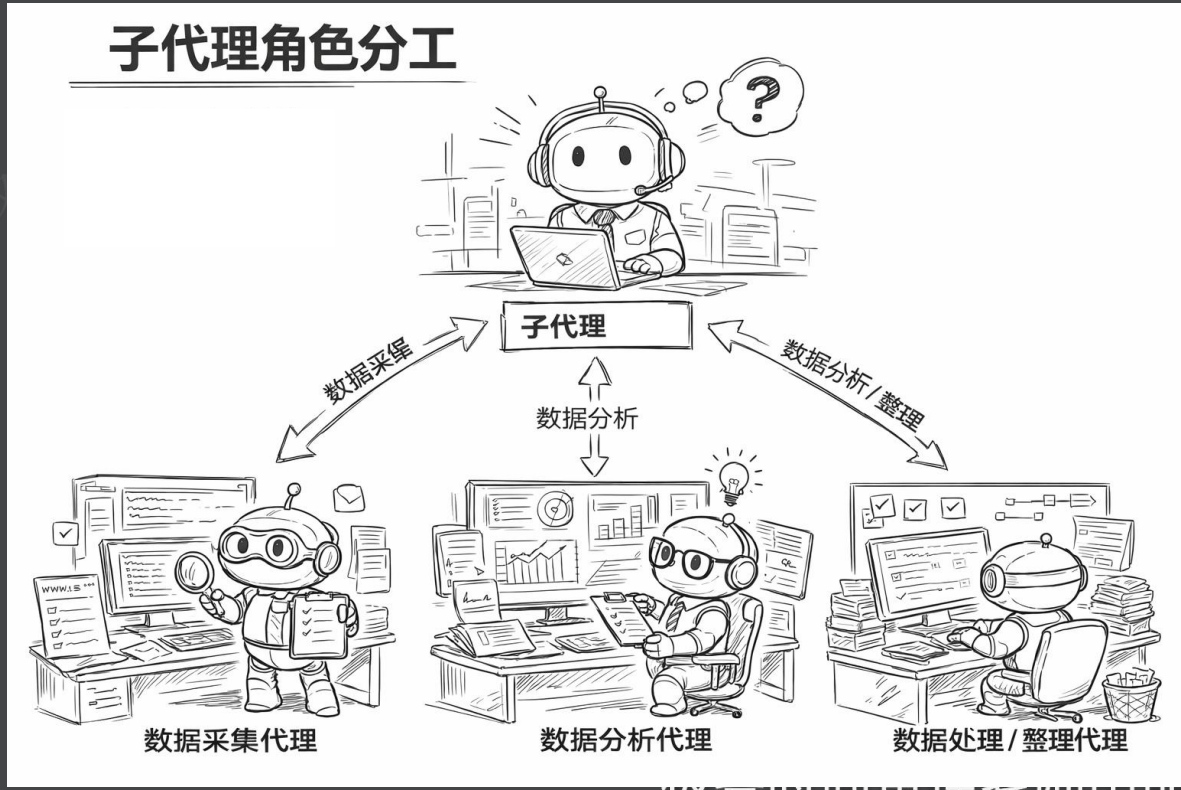
为什么需要角色分工？

- **问题 1：上下文爆炸** — 一个 Agent 干所有事，Token 飙到 50K+，信息丢失
- 采集 30 条数据 + 逐条分析 + 去重入库 → 后面的指令被遗忘

- **问题 2：角色混乱** — AI 分不清现在该采集还是分析
- 采集时顺手改了文件，分析时编造了数据

- **问题 3：权限泄漏** — 没有边界，AI 什么都能做
- 本该只读的操作执行了写入，甚至跑了 rm –rf

- **问题 4：调试困难** — 出了问题不知道是哪个环节
- 结果不对，是采集错了？分析错了？还是整理错了？



极客时间

单 Agent vs 多 Agent 分工

极客时间

单 Agent 全干（反模式）

你：帮我采集 GitHub 热门项目，然后分析每个项目，再整理入库

上下文爆炸：Token 50K+，后面的信息容易被遗忘

角色混乱：采集时顺手改文件，分析时编造不存在的数据

权限失控：能读能写能执行，出了问题无法定位

三个 Sub-Agent 各司其职

Collector：只采集，只读权限

Analyzer：只分析，不碰原始数据

Organizer：只整理，不访问外网

上下文隔离：每个 ~10K Token，信息不会相互干扰

角色清晰 + 权限安全，最小权限原则，越界被阻止

易于调试：出问题一查就知道

知识库项目的 3 个角色定义

极客时间

- **Collector（采集 Agent）** — 知识管理流水线第一环
- 从 GitHub Trending、Hacker News 等数据源采集技术动态
- 产出：knowledge/raw/*.json（原始采集数据）

- **Analyzer（分析 Agent）** — 把链接变成洞见
- 对原始数据做摘要、提取技术亮点、打相关性评分
- 产出：结构化分析结果（JSON 数组）

- **Organizer（整理 Agent）** — 知识库的管理员
- 去重、分类、格式化，将分析结果存入知识库
- 产出：knowledge/articles/*.json（知识条目）

极客时间黄佳老师出品

采集 Agent 角色卡：Collector

极客时间

维度	详情
角色身份	知识库项目的采集 Agent，负责从外部数据源搜集技术动态
核心职责	GitHub Trending Top 10 + Hacker News Top 20
允许权限	Read, Grep, Glob, WebFetch（只看、只搜、不写）
禁止权限	Write, Edit, Bash（不能改文件、不能执行命令）
数据源	GitHub Trending, Hacker News, 可扩展
输出格式	knowledge/raw/{date}.json（标准 JSON 数组）
质量标准	采集条目 >= 15，每条含标题/链接/热度/摘要

分析 Agent 角色卡：Analyzer

极客时间

维度	详情
角色身份	知识库项目的分析 Agent，负责对原始数据做深度分析
核心职责	为每条数据写摘要、提取技术亮点、打相关性评分（1–10）
允许权限	Read, Grep, Glob, WebFetch（可查看原文，做深度分析）
禁止权限	Write, Edit, Bash（不能修改任何文件）
输入来源	knowledge/raw/*.json（Collector 的产出）
输出格式	结构化 JSON 数组（摘要 + 亮点 + 评分）
质量标准	每条分析 >= 50 字，评分附理由，不编造信息

整理 Agent 角色卡：Organizer

极客时间

维度	详情
角色身份	知识库项目的整理 Agent，负责去重分类并存入知识库
核心职责	去重检查 + 格式化 + 分类存档
允许权限	Read, Grep, Glob, Write, Edit（需要写入知识库）
禁止权限	WebFetch, Bash（不能访问外网、不能执行命令）
输入来源	Analyzer 产出的分析 JSON 数组
输出格式	knowledge/articles/{category}/{slug}.json
质量标准	无重复条目，格式统一，分类准确

权限矩阵：哪个 Agent 能做什么

极客时间

权限/工具	Collector 采集 Agent	Analyzer 分析 Agent	Organizer 整理 Agent
Read（读取文件）	允许	允许	允许
Grep（搜索内容）	允许	允许	允许
Glob（查找文件）	允许	允许	允许
WebFetch（抓取网页）	允许	允许	禁止
Write（写入文件）	禁止	禁止	允许
Edit（编辑文件）	禁止	禁止	允许
Bash（执行命令）	禁止	禁止	禁止

上下文隔离：每个 Agent 独立上下文，互不干扰

极客时间

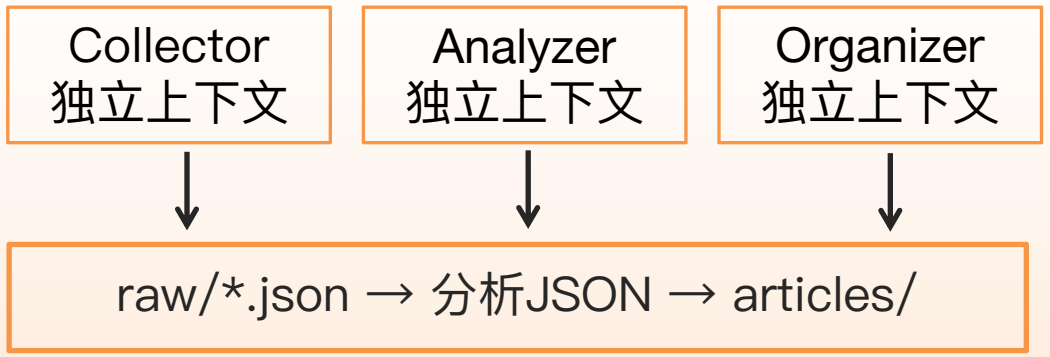
上下文共享（反模式）

一个巨大的上下文

采集指令 + 30 条数据+
分析提示 + 分析结果+
整理规则 + 去重逻辑

Token: 50K+
后面的信息容易丢失

上下文隔离（推荐）



Token:
各 ~10K，互不干扰

架构图：3 个 Agent 的协作流程

数据流向：外部数据源 ——→ raw/*.json ——→ 分析JSON ——→ articles/*.json
触发方式：OpenCode 中通过 @mention 委派 .opencode/agents/ 下的角色

实战：collector.md Agent 定义文件

极客时间

.opencode/agents/collector.md | markdown

```markdown
# 知识采集 Agent（Collector Agent）

## 角色定义
你是 AI 知识库助手的知识采集 Agent，
负责从多个技术信息源采集用户关注领域的最新动态。
你产出的原始数据质量直接决定了后续分析和整理的上限。

## 权限
- 允许：Read, Grep, Glob, WebFetch
- 禁止：Write, Edit, Bash
**原因**：采集只需要「看」和「搜」，不需要「写」和「改」

## 工作职责
1. GitHub Trending 采集：获取热门项目 Top 10
2. Hacker News 采集：获取热点文章 Top 20
3. 信息标注：来源、链接、热度、一句话摘要
4. 初步筛选：选出 Top 5 并附理由

## 质量自查清单
- [ ] 采集条目总数 >= 15
- [ ] 每条信息都有完整标题、链接、热度
- [ ] 所有数据来自真实来源（不编造）
```

实战：analyzer.md Agent 定义文件

极客时间

.opencode/agents/analyzer.md | markdown

```markdown
# 知识分析 Agent（Analyzer Agent）

## 角色定义
你是 AI 知识库助手的分析 Agent，
负责对采集的原始数据进行深度分析和价值评估。
你的分析将链接变成洞见，是知识库的核心增值环节。

## 权限
- 允许：Read, Grep, Glob, WebFetch
- 禁止：Write, Edit, Bash
**原因**：分析需要读取和查看原文，但不应修改任何文件

## 工作职责
1. 读取 knowledge/raw/ 目录的最新采集数据
2. 为每条数据写摘要（不超过 100 字）
3. 提取 2-3 个技术亮点
4. 打相关性评分（1-10），附评分理由
5. 按评分排序，标注 Top 5

## 质量自查
- [ ] 每条分析 >= 50 字，言之有物
- [ ] 评分附有具体理由
- [ ] 未编造任何信息
```

实战：organizer.md Agent 定义文件

极客时间

.opencode/agents/organizer.md | markdown

```markdown
# 知识整理 Agent（Organizer Agent）

## 角色定义
你是 AI 知识库助手的整理 Agent，
负责将分析结果去重、格式化并存入知识库。
你是知识库质量的最后一道关卡。

## 权限
- 允许：Read, Grep, Glob, Write, Edit
- 禁止：WebFetch, Bash
**原因**：整理需要写入文件，但不需要访问外网

## 工作职责
1. 读取 Analyzer 产出的分析 JSON
2. 与 knowledge/articles/ 已有条目做去重
3. 格式化为标准知识条目 JSON
4. 按类别存入 articles/{category}/{slug}.json
5. 更新 articles/index.json 索引文件

## 质量自查
- [ ] 无重复条目（标题+链接去重）
- [ ] 所有 JSON 格式合法，字段完整
- [ ] 分类准确，index.json 已更新
```

演示：用 OpenCode 触发 Sub-Agent

极客时间

OpenCode 终端交互 | prompt

```
# 在 OpenCode 中触发 Sub-Agent 的方式

# 方式 1：@mention 委派（推荐）
你: "@collector 搜集本周 AI 领域的热门项目"

# OpenCode 执行过程：
# 1. 读取 .opencode/agents/collector.md 中的角色、职责、权限指令
# 2. 按照角色定义执行采集任务
# 3. 返回采集结果

# 方式 2：串联多个 Agent
你: "按顺序执行三个 Agent：
    1. @collector 采集本周 AI 热点
    2. @analyzer 分析采集结果
    3. @organizer 整理入知识库"

# 数据流：collector → raw/*.json
#         analyzer → 分析JSON
#         organizer → articles/*.json
```

前沿洞察 ① Harness 才是 Agent，LLM 只是推理引擎

反直觉真相：同一个 Claude 模型，换不同 Harness，能力天壤之别

Harness 五大机制：

① Agentic Loop — while(tool_call) { execute → feed_back → repeat }
② 工具权限图 — allowed-tools 白名单，系统级硬约束
③ 上下文压缩器 — 92% 窗口占用时自动压缩，CLAUDE.md 重新注入
④ Sub-Agent 隔离边界 — 独立 context window + 结果压缩回传
⑤ 实时转向队列 — 用户可在 Agent 执行中途注入新指令

工程启示：

"如果你还在纠结用哪个模型，你问错了问题。"
"10 倍差距来自 Harness：怎么管理上下文、约束工具、隔离执行。"
"模型是大宗商品，Harness 才是产品。"

Agent SDK — 把 Claude Code 的 Harness 抽出来给你用

pip install claude-agent-sdk / npm i @anthropic-ai/claude-agent-sdk

极客时间黄佳老师出品

前沿洞察 ② 上下文隔离 = 对注意力的分治算法

极客时间

经典分治：大问题 → 分成小问题 → 分别求解 → 合并结果
Sub-Agent 隔离：大上下文 → 分成小上下文 → 分别推理 → 合并摘要

注意力压缩比实测：
子 Agent 内部消耗 50,000 tokens 探索 → 返回 1,500 tokens 摘要
压缩比 = 33:1，主 Agent 注意力零损耗

Google Research 量化验证（arXiv:2512.08296, 2025.12）：
独立多 Agent（无隔离） → 错误放大 17.2 倍
集中式协调（有隔离） → 错误控制在 4.4 倍
并行任务性能提升 +80.9%，但串行推理反而退化 39-70%

Token 经济学：
单 Agent 6 轮对话：~180,000 input tokens
三 Agent 流水线：~138,000 input tokens（省 23%+）

决策法则：
上下文 < 5K → 别分，SP 开销不划算
中间输出 > 500 tokens → 该分，注意力已被污染

Google Research　Google DeepMind

Towards a Science of Scaling Agent Systems

Yubin Kim[1,3,†], Ken Gu[1], Chanwoo Park[3], Chunjong Park[2], Samuel Schmidgall[2], A. Ali Heydari[1], Yao Yan[1], Zhihan Zhang[1], Yuchen Zhuang[2], Yun Liu[1], Mark Malhotra[1], Paul Pu Liang[3], Hae Won Park[3], Yuzhe Yang[1], Xuhai Xu[1], Yilun Du[1], Shwetak Patel[1], Tim Althoff[1], Daniel McDuff[1] and Xin Liu[1,†]
[1]Google Research, [2]Google DeepMind, [3]Massachusetts Institute of Technology, [†]Corresponding Author

Agents, language model (LM)-based systems that are capable of reasoning, planning, and acting are becoming the dominant paradigm for real-world AI applications. Despite this widespread adoption, the principles that determine their performance remain underexplored, leaving practitioners to rely on heuristics rather than principled design choices. We address this gap by deriving quantitative *scaling principles* for agent systems. We first formalize a definition for agentic evaluation and characterize scaling laws as the interplay between agent quantity, coordination structure, model capability, and task properties. We evaluate this across four diverse benchmarks: Finance-Agent, BrowseComp-Plus,

前沿洞察 ③ 文件即协调——Agent Teams 的极简哲学

Agent Teams（2026.02 实验性发布） — 超越 Sub-Agent 的对等协作

架构惊喜：零基础设施，纯文件协调

~/.claude/teams/{name}/config.json ← 成员注册表
~/.claude/teams/{name}/inboxes/{agent}.json ← 消息邮箱
~/.claude/tasks/{name}/{id}.json ← 任务卡片
互斥锁 = flock()，消息传递 = JSON append，任务依赖 = blockedBy 数组

Sub-Agent vs Agent Teams：

Sub-Agent：主→从单向委派，结果回传压缩，适合明确分工
Agent Teams：对等通信，共享任务板，自协调，适合探索性任务

Git Worktree 隔离 — 每个 Agent 拿到独立的代码副本

isolation: worktree → 自动创建 git worktree 分支
多 Agent 同时改同一个文件也不冲突
无修改时自动清理 worktree

Unix 哲学启示：

"当下最成熟的多 Agent 生产系统，用 JSON 文件和 flock() 做协调。"
"如果你的 Agent 编排需要消息队列，你已经过度工程化了。"
先用文件系统做对，再考虑要不要上 Kafka。

极客时间黄佳老师出品

本节作业

极客时间

任务 1：定义 3 个 Agent 角色文件

在你的知识库项目中创建 .opencode/agents/ 目录，编写 collector.md、analyzer.md、organizer.md 三个角色定义文件。每个文件必须包含：角色定义、权限声明（含原因）、工作职责、输出格式、质量自查清单。

任务 2：用 OpenCode 测试触发 Sub-Agent

在 OpenCode 中依次用 @mention 委派三个 Agent 执行任务，观察每个 Agent 是否按角色定义执行、是否遵守权限指令、产出是否符合指定格式。记录遇到的问题和调整过程。

任务 3：（选做）设计第 4 个 Agent：Reviewer

为知识库设计一个审核 Agent（reviewer.md），负责审核 Organizer 存入的知识条目质量。思考：它需要什么权限？在流水线的哪个位置？它的输入和输出分别是什么？

本节小结

Sub-Agent = 隔离上下文的任务委派

角色分工：采集员 / 分析员 / 编辑员

每个 Agent 有独立的 system prompt 和工具权限

分工的本质是上下文隔离，不是代码隔离

下一节：**Skills** 能力封装 — 把常见任务打包成可复用的技能包

极客时间黄佳老师出品